



INTERNAL ENGINEERING REFERENCE

Large Language Models

A Technical Handbook — Architecture, Training, Inference, Retrieval-Augmented Generation, and Production Deployment

Prepared for: Engineering & AI Technology Team (Software Engineers, ML/AI Engineers, Data Scientists)

Document type: Technical Reference Handbook

Version: 1.0 | **Date:** July 2026

Transformers

Tokenization

Pretraining & RLHF

Inference & Decoding

RAG

Evaluation

Production Best Practices

Table of Contents

A field reference to how LLMs work internally and how to build production systems around them.

Chapter 1 — Introduction to Large Language Models	3
1.1 From Artificial Intelligence to Large Language Models	3
1.2 The World Before LLMs	3
1.3 Birth of Large Language Models — A Timeline	4
1.4 What Is an LLM?	5
Chapter 2 — The Transformer Revolution	6
2.1 Why Transformers Were Invented	6
2.2 "Attention Is All You Need" (2017)	7
2.3 Transformer Architecture — Deep Dive	8
Chapter 3 — Anatomy of an LLM	11
3.1 Tokenization: BPE, WordPiece, SentencePiece	11
3.2 Embeddings, Vocabulary & Parameters	12
3.3 Hidden States, Attention Heads & Context Window	12
3.4 KV Cache, Logits, Softmax & Next-Token Prediction	13
3.5 The Complete Inference Pipeline	14
Chapter 4 — Building and Using an LLM	15
4.1 Building: Data, Pretraining & Fine-Tuning	15
4.2 Alignment: SFT, RLHF, DPO, LoRA / QLoRA	16
4.3 Inference: Decoding, Temperature, Top-K/Top-P, Streaming	17
4.4 Practical Usage: Prompting, Function Calling, Structured Output	18
Chapter 5 — RAG and Enterprise LLM Applications	19
5.1 Why Retrieval-Augmented Generation?	19
5.2 The RAG Pipeline	19
5.3 Chunking, Embeddings & Vector Databases	20
5.4 Hybrid Search, Re-ranking & Context Compression	21
5.5 Agentic RAG	21
Chapter 6 — Evaluation, Deployment & Best Practices	22
6.1 Evaluation Metrics (Traditional, LLM/RAG-specific, Performance)	22
6.2 Production Best Practices & Security	23

Introduction to Large Language Models

1.1 From Artificial Intelligence to Large Language Models

Artificial Intelligence progressed through distinct eras: **symbolic AI** (hand-coded logic and expert systems, 1950s–1980s), **statistical machine learning** (feature-engineered models such as SVMs and decision trees, 1990s–2000s), and **deep learning** (neural networks that learn features automatically, 2010s onward). Natural Language Processing followed the same arc — from grammar rules, to statistical n-gram models, to neural sequence models, to today's Transformer-based Large Language Models.

A **language model** is any system that assigns a probability to a sequence of words, or predicts the next word given the previous ones. Early language models were narrow and task-specific. As datasets, compute, and neural architectures scaled — particularly after the 2017 Transformer — language models grew large enough to generalize across nearly any language task from a single set of weights. That scale is what turned "a language model" into a **Large Language Model**: a shift in degree that produced a shift in kind, unlocking few-shot reasoning, instruction-following, and emergent capabilities that smaller models did not exhibit.

1.2 The World Before LLMs

Before LLMs, NLP was a patchwork of specialized techniques, each addressing a narrow slice of language understanding:

RULE-BASED SYSTEMS

Hand-written grammars and pattern-matching rules (e.g., ELIZA). Brittle, exhaustive to maintain, and unable to generalize beyond the rules authored.

RNN / LSTM LANGUAGE MODELS

Recurrent networks processed text sequentially, carrying a hidden state forward. LSTMs/GRUs mitigated — but did not solve — vanishing gradients and long-range dependency loss.

KEYWORD MATCHING & SEARCH ENGINES

Early search and FAQ bots matched literal keywords or TF-IDF scores rather than meaning — "car" and "automobile" were unrelated tokens.

EARLY VOICE ASSISTANTS

Siri, Alexa, and Google Assistant (pre-2018) combined intent classifiers, slot-filling, and rigid dialogue trees — effective for narrow commands, poor at open-ended conversation.

STATISTICAL NLP & N-GRAM MODELS

Predicted the next word from fixed-size word-history counts (bigrams, trigrams). Simple and fast, but suffered from data sparsity and no real understanding of long-range context.

LIMITATIONS, IN SUMMARY

No shared world knowledge, no transfer across tasks, weak handling of ambiguity/context, and heavy manual feature engineering per task.

Before LLMs	After LLMs
Keyword matching	Context understanding
Task-specific models	General-purpose models
Separate model for each task	One model for multiple tasks
Limited conversations	Natural, multi-turn conversations
Rule-based responses	Context-aware reasoning
Static knowledge pipelines	Dynamic, prompt-based interactions

1.3 Birth of Large Language Models — A Timeline

2017	"Attention Is All You Need" Google Research introduces the Transformer, replacing recurrence with self-attention and enabling full parallelization of sequence training.
2018	BERT Google's bidirectional encoder popularizes large-scale pretraining + fine-tuning for language understanding tasks.
2018	GPT-1 OpenAI shows that a decoder-only Transformer pretrained on raw text, then fine-tuned, transfers well across NLP tasks.
2019	GPT-2 Scaling to 1.5B parameters produces surprisingly coherent long-form generation and strong zero-shot behavior.
2020	GPT-3 175B parameters demonstrate few-shot, in-context learning — the model adapts to new tasks from examples in the prompt alone, no gradient update required.
2022	ChatGPT Instruction tuning + RLHF turn a raw completion model into a helpful, safe conversational assistant, driving mainstream adoption.
2023	Llama, Mistral Efficient, openly-released model families accelerate research and self-hosted / on-prem enterprise deployment.
2024–2026	Multimodal & Reasoning Models Unified text/image/audio models and models trained with extended reasoning (chain-of-thought reinforcement) push performance on coding, math, and agentic tasks.

Why 2017 Was the Turning Point

Every model above 2018 — BERT, GPT, Llama, Mistral, Claude, Gemini — is a variant of the same architecture published in 2017. Recurrent models processed tokens one-by-one, so training time scaled with sequence length and GPUs were never fully utilized. The Transformer removed recurrence entirely: every token attends to every other token in parallel matrix operations. That single change made it economically feasible to train on internet-scale corpora using thousands of GPUs at once — scale is what turned language models into *Large* Language Models.

1.4 What Is an LLM?

A **Large Language Model** is a neural network — almost always a Transformer — with billions to trillions of learned parameters, trained on massive text (and increasingly multimodal) corpora to predict the next token in a sequence. Despite the simplicity of that training objective, next-token prediction at scale forces the model to internalize grammar, facts, reasoning patterns, coding conventions, and style.

WHY "LARGE"

Parameter counts range from ~1B (edge-deployable) to 1T+ (frontier models). Larger models, trained on more data with more compute, reliably exhibit lower loss and new emergent capabilities — a relationship described by neural scaling laws.

PARAMETERS

The learned weights — matrices in attention and feed-forward layers — that encode everything the model "knows." They are adjusted during training and frozen at inference.

CAPABILITIES

Text generation, summarization, translation, question answering, reasoning, code generation, tool/function calling, and multi-turn dialogue — all from one general-purpose model.

Limitations to Design Around

LLMs hallucinate confidently, have a fixed knowledge cutoff, are sensitive to prompt phrasing, cannot natively access private or real-time data without tools/RAG, and incur non-trivial latency and inference cost at scale. Chapters 5 and 6 cover the engineering patterns that mitigate each of these.

Typical enterprise applications: customer-support copilots, internal knowledge assistants (RAG), code generation and review, document summarization/extraction, semantic search, and autonomous or semi-autonomous agents that call tools and APIs.

The Transformer Revolution

2.1 Why Transformers Were Invented

Before 2017, sequence modeling was dominated by Recurrent Neural Networks (RNNs) and their gated variants (LSTM, GRU). These architectures processed a sequence one token at a time, updating a hidden state as they went. This design created four compounding problems:

SEQUENTIAL COMPUTATION

Token t could not be processed until token $t-1$ finished. Training could not be parallelized across the time dimension, making it slow to scale on GPUs/TPUs.

LONG-TERM DEPENDENCY LOSS

Even with LSTM gating, information from far-earlier tokens was diluted by the time it reached later steps — a document's opening sentence rarely influenced its conclusion.

VANISHING / EXPLODING GRADIENTS

Backpropagating error across hundreds of time steps caused gradients to shrink toward zero (or blow up), making early tokens in long sequences effectively unlearnable.

LACK OF PARALLELIZATION

The direct consequence of sequential state: training throughput scaled poorly, capping practical model and dataset size — the exact ingredients LLMs need in abundance.

The Transformer's answer was to **discard recurrence entirely** and replace it with **self-attention**: every position in a sequence directly computes a relationship to every other position through matrix multiplication — an operation that is trivially parallelizable across an entire sequence and scales cleanly with more GPUs.

2.2 "Attention Is All You Need" (2017)

Google Brain / Google Research, 2017

Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin

The paper was motivated by a specific engineering bottleneck: existing sequence-to-sequence models (RNN/LSTM encoder-decoders, even with attention layers bolted on) were accurate but too slow to train on the amount of data the authors believed language modeling would eventually require. The core question was whether attention — until then, an auxiliary mechanism added to recurrent models — could stand entirely on its own.

MAIN CONTRIBUTIONS

- **Self-attention as the primary computation**, replacing recurrence and convolution for sequence transduction.
- **Multi-head attention**, allowing the model to jointly attend to information from different representation subspaces at different positions.
- **Positional encoding**, injecting order information without recurrence.
- A fully parallelizable encoder-decoder stack that trained dramatically faster on the same hardware and reached a new state of the art in machine translation.

Why It Revolutionized NLP

Parallel training meant researchers could, for the first time, profitably trade compute and data for quality by simply scaling up — more layers, more parameters, more tokens. That scaling recipe, not any single clever trick, is what produced BERT, GPT, and every subsequent LLM. The Transformer decoder in particular became the near-universal backbone: GPT, Llama, Mistral, Claude, and Gemini are all, at their core, stacks of the same self-attention + feed-forward block described in this paper.

2.3 Transformer Architecture — Deep Dive

The original paper describes an **encoder-decoder** architecture built from repeating blocks. Encoders are used for understanding tasks (e.g., BERT); decoders — used alone, autoregressively — power modern generative LLMs (GPT-family, Llama, Claude). Both share the same core components.

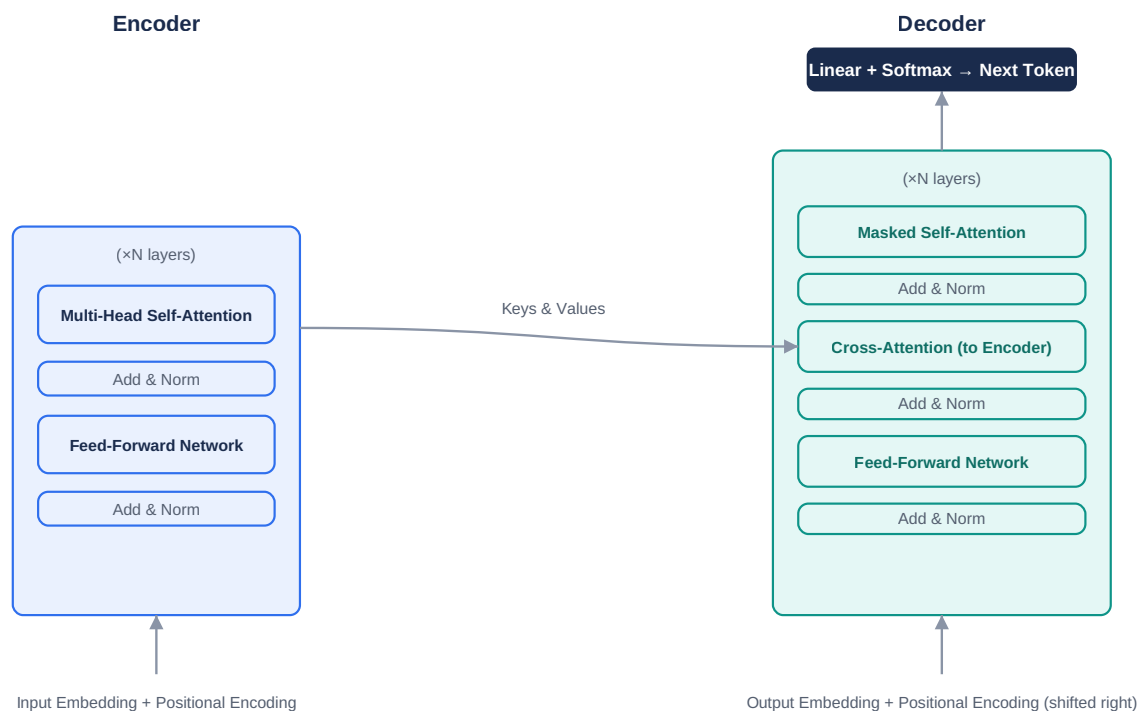


Figure 2.1 — Original encoder-decoder Transformer. Decoder-only LLMs (GPT, Llama, Claude) drop the encoder and cross-attention, keeping masked self-attention + FFN blocks stacked N times.

Self-Attention: Query, Key, Value

Self-attention lets every token look at every other token and decide how much to "attend" to each. Each token's embedding is projected into three vectors via learned weight matrices:

- **Query (Q)** — what this token is "looking for."
- **Key (K)** — what this token "offers" as a label for matching.
- **Value (V)** — the actual content passed on if attention is paid to this token.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

The dot product $Q \cdot K^T$ scores how relevant every other token is to the current one; dividing by $\sqrt{d_k}$ (the key dimension) keeps gradients stable; softmax turns scores into a probability distribution; multiplying by V produces a context-aware blend of every token's value, weighted by relevance.

Multi-Head Attention

Rather than computing attention once, the model runs h attention "heads" in parallel, each with its own learned $Q/K/V$ projections into a smaller subspace. One head might learn to track syntactic dependencies, another coreference, another local word order. The outputs of all heads are concatenated and linearly projected back to the model dimension — giving the model several independent "views" of the same sequence simultaneously.

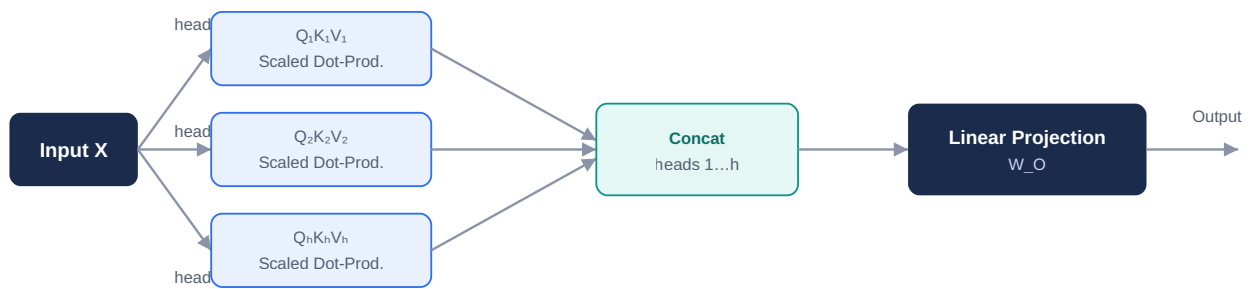


Figure 2.2 — Multi-head attention: h parallel attention computations, concatenated and re-projected.

Positional Encoding

Because self-attention has no inherent notion of order (it treats input as a set, not a sequence), position must be injected explicitly. The original paper adds fixed sine/cosine signals of varying frequency to each token embedding; modern LLMs commonly use **Rotary Positional Embeddings (RoPE)**, which rotate Q/K vectors as a function of position and generalize better to longer sequences than sinusoidal or learned absolute encodings.

Feed-Forward Networks, Residuals & Layer Normalization

Feed-Forward Network (FFN): After attention mixes information across tokens, a position-wise two-layer MLP (typically expanding to $4\times$ the model dimension with a non-linearity, then projecting back) processes each token independently, adding representational capacity.

Residual Connections: Each sub-layer's output is added back to its input ($x + \text{Sublayer}(x)$), giving gradients a direct path through the network and making very deep stacks (dozens to hundreds of layers) trainable.

Layer Normalization: Normalizes activations across the feature dimension for each token, stabilizing training. Modern LLMs typically apply **Pre-LN** (normalize before the sub-layer) rather than the original Post-LN, for more stable large-scale training.

Together, Attention $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm forms one **Transformer block**; a modern LLM stacks dozens to over a hundred of these blocks.

COMPLETE TRANSFORMER WORKFLOW

Text $\rightarrow$ Tokens $\rightarrow$ Token + Positional Embeddings $\rightarrow$ [Self-Attention $\rightarrow$ Add & Norm $\rightarrow$ Feed-Forward $\rightarrow$ Add & Norm] $\times$ N layers $\rightarrow$ Final Linear Layer $\rightarrow$ Softmax $\rightarrow$ Probability distribution over the vocabulary for the next token.

Anatomy of an LLM

This chapter walks through every stage a raw string of text passes through on its way to becoming a generated response — the vocabulary every engineer working with LLM APIs or infrastructure should know cold.

3.1 Tokenization: BPE, WordPiece, SentencePiece

Neural networks operate on numbers, not characters. **Tokenization** is the process of splitting raw text into subword units — **tokens** — and mapping each to an integer **Token ID** from a fixed vocabulary (typically 32K–200K entries). Subword tokenization strikes a balance: common words become a single token, rare or unseen words decompose into smaller known pieces, so the model never encounters a truly "unknown" word.

Method	Approach	Used By
BPE (Byte-Pair Encoding)	Iteratively merges the most frequent adjacent character/byte pairs into new vocabulary entries.	GPT-family, Llama
WordPiece	Similar to BPE but merges pairs that maximize training-data likelihood rather than raw frequency.	BERT
SentencePiece	Language-agnostic; treats text as a raw stream (including spaces) so it needs no pre-tokenization by whitespace — useful for non-space-delimited languages.	Llama, T5, many multilingual models

Rule of Thumb

In English, one token $\approx$ 4 characters $\approx$ $\frac{3}{4}$ of a word. "Tokenization" itself might split into **Token** + **ization**. This directly determines API billing (most providers price per token) and how much text fits in a context window.

3.2 Embeddings, Vocabulary & Parameters

Each Token ID indexes into an **embedding matrix** of shape `[vocab_size × d_model]`, producing a dense vector (e.g., 4096 dimensions) that captures learned semantic and syntactic properties — similar words end up with similar vectors. This embedding matrix, together with every weight in every attention and feed-forward layer, makes up the model's **parameters**. A "70B model" has 70 billion such learned numbers, each a 16- or 32-bit float, adjusted during training and frozen at inference.

3.3 Hidden States, Attention Heads & Context Window

Hidden states are the intermediate vector representations of each token as it passes through successive Transformer blocks — each layer refines the previous layer's hidden state, progressively encoding richer context.

Attention heads are the parallel self-attention computations within each block (Section 2.3); a large model may have 32–128 heads per layer across 32–120+ layers.

Context window is the maximum number of tokens (input + output combined) the model can process in a single call — ranging from 4K in early models to 200K–1M+ in current frontier models. Everything the model "knows" about the current conversation must fit inside this window; nothing outside it is visible.

3.4 KV Cache, Logits, Softmax & Next-Token Prediction

- **KV Cache:** During autoregressive generation, each new token only needs to attend to previous tokens' Key/Value vectors — which don't change. Caching them avoids recomputing attention over the full sequence at every step, turning an $O(n^2)$ per-token cost into an amortized $O(n)$ — the single most important inference optimization in production LLM serving.
- **Logits:** The raw, unnormalized output scores the model assigns to every token in the vocabulary for "what comes next" — one score per vocabulary entry, before any probability conversion.

- **Softmax:** Converts logits into a valid probability distribution (all values between 0 and 1, summing to 1) over the vocabulary.
- **Next-Token Prediction:** A token is sampled (or greedily selected) from this distribution, appended to the sequence, and the process repeats — one token at a time — until an end-of-sequence token or length limit is reached.

3.5 The Complete Inference Pipeline

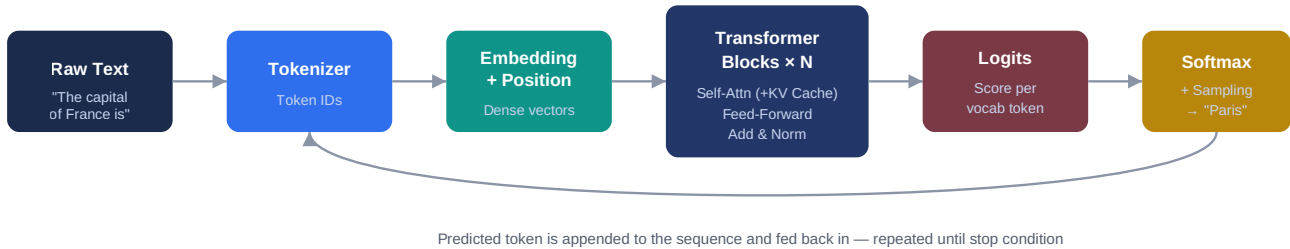


Figure 3.1 — End-to-end autoregressive inference pipeline, one generated token per full pass.

This loop — encode context, predict one token, append it, repeat — is why LLM output streams token-by-token and why longer generations cost proportionally more latency and compute.

Building and Using an LLM

4.1 Building: Data, Pretraining & Fine-Tuning

DATA COLLECTION

Web crawls, books, code repositories, academic papers, and licensed datasets — assembled into a corpus ranging from hundreds of billions to tens of trillions of tokens.

TOKENIZATION

The cleaned corpus is tokenized once using the chosen vocabulary (Section 3.1) and stored in an efficient binary format for fast streaming during training.

DATA CLEANING

Deduplication, quality filtering, PII/toxicity removal, language identification, and near-duplicate detection. Data quality has a larger effect on downstream performance than raw volume.

FOUNDATION MODELS

The output of pretraining — a general-purpose base model with broad knowledge but no particular conversational behavior. Foundation models are the substrate every downstream fine-tune builds on.

Pretraining trains the Transformer from randomly initialized weights on the full corpus using the simple next-token-prediction objective, at massive scale — thousands of GPUs/TPUs running for weeks to months. This single, self-supervised objective (no manual labels needed) is what teaches the model grammar, facts, reasoning patterns, and coding ability.

4.2 Alignment: SFT, RLHF, DPO, LoRA / QLoRA

A freshly pretrained base model is good at continuing text, not at following instructions or refusing unsafe requests. **Alignment** is the set of post-training stages that shape raw completion ability into a helpful, instructable assistant.

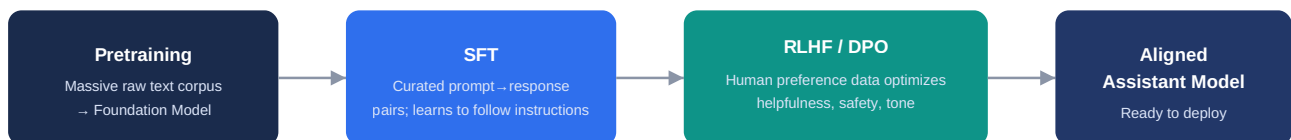


Figure 4.1 — Post-training alignment pipeline from base model to deployable assistant.

Technique	What It Does
SFT (Supervised Fine-Tuning)	Trains on curated instruction → response examples so the model learns the "chat" behavior and task-following format.
RLHF (Reinforcement Learning from Human Feedback)	Humans rank multiple model outputs; a reward model is trained on those rankings; the LLM is then optimized (typically via PPO) to maximize predicted reward.
DPO (Direct Preference Optimization)	A simpler alternative to RLHF that optimizes directly on preference pairs (chosen vs. rejected response) without training a separate reward model — cheaper and more stable to run.
LoRA / QLoRA	Parameter-efficient fine-tuning: freezes the base model and trains small low-rank adapter matrices injected into each layer (QLoRA additionally quantizes the frozen base to 4-bit). Enables fine-tuning multi-billion-parameter models on a single GPU.

When to Reach for Each

Full RLHF is how frontier labs align flagship models. For enterprise customization — teaching a model your product's tone, domain vocabulary, or output schema — LoRA/QLoRA fine-tuning on a few thousand curated examples is typically far cheaper and sufficient; for most knowledge-injection needs, RAG (Chapter 5) is cheaper still and requires no training at all.

4.3 Inference: Decoding, Temperature, Top-K / Top-P, Streaming

At inference, the model computes a probability distribution over the vocabulary at each step (Section 3.4); **decoding strategy** determines how the actual next token is chosen from that distribution.

Greedy decoding always picks the single highest-probability token — deterministic but often repetitive.

Temperature rescales the logits before softmax. Low temperature (≈ 0.1 – 0.3) sharpens the distribution toward the most likely tokens (focused, deterministic); high temperature (≈ 0.8 – 1.2 +) flattens it (more diverse, more creative, higher risk of incoherence).

Top-K sampling restricts sampling to the K highest-probability tokens, discarding the long tail.

Top-p / Nucleus sampling restricts sampling to the smallest set of tokens whose cumulative probability exceeds p (e.g., 0.9) — adapting the candidate pool size to how confident the model is at each step, unlike the fixed-size Top-K.

Streaming: Because generation is inherently one-token-at-a-time (Figure 3.1), production APIs stream tokens to the client as they're produced rather than waiting for the full response — dramatically improving perceived latency for chat interfaces.

4.4 Practical Usage: Prompting, Function Calling, Structured Output

- **Prompt Engineering:** The practice of designing instructions, examples (few-shot demonstrations), and context structure to reliably steer model output — the highest-leverage, lowest-cost lever for improving quality, since it requires no training.
- **Function / Tool Calling:** The model is given a schema of available functions (name, description, parameters); instead of answering directly, it can emit a structured request to call one, receive the result, and continue reasoning — the mechanism underlying agents and tool-using assistants.
- **Structured Outputs:** Constraining generation to a defined JSON schema (via grammar-constrained decoding or schema-aware prompting) so downstream code can parse responses reliably without brittle string parsing.

RAG and Enterprise LLM Applications

5.1 Why Retrieval-Augmented Generation?

LLMs know only what was in their training data, frozen at a knowledge cutoff, and have no access to private, proprietary, or real-time information by default. **Retrieval-Augmented Generation (RAG)** solves this without retraining the model: relevant documents are retrieved from an external knowledge source at query time and injected into the prompt as context, grounding the model's answer in current, verifiable, organization-specific data — and reducing hallucination versus relying on parametric memory alone.

5.2 The RAG Pipeline

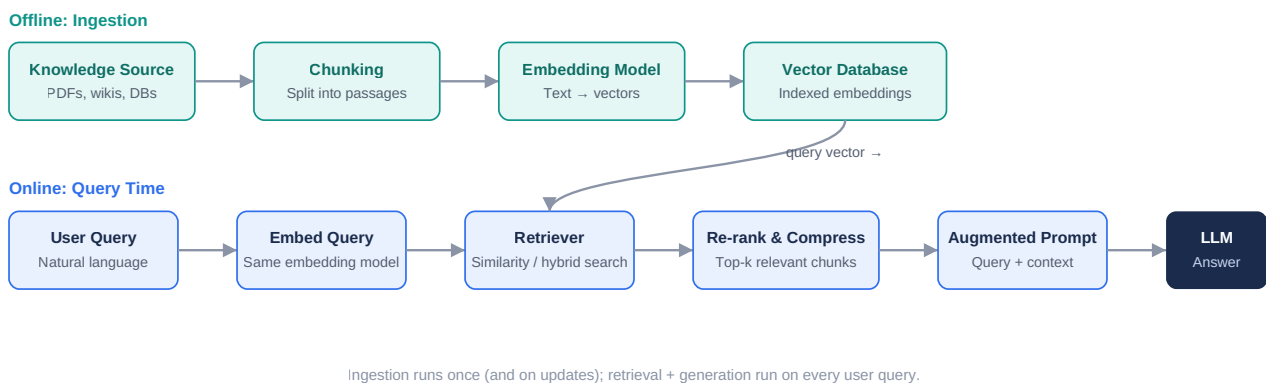


Figure 5.1 — The end-to-end RAG pipeline: offline document ingestion feeds a vector database that is queried live at inference time.

5.3 Chunking, Embeddings & Vector Databases

Chunking splits source documents into passages small enough to embed meaningfully and fit inside retrieved context, but large enough to preserve coherent meaning — typically 200–800 tokens, often with overlap between consecutive chunks to avoid severing ideas at boundaries. Chunking strategy (fixed-size, sentence/paragraph-aware, or semantic) materially affects retrieval quality.

Embeddings (produced by models such as MiniLM, BGE, or E5) map each chunk to a dense vector such that semantically similar text lands close together in vector space, enabling meaning-based rather than keyword-based search.

Vector databases (Chroma, Pinecone, FAISS, Weaviate, pgvector, and others) index millions of embeddings for fast approximate nearest-neighbor search, returning the most semantically relevant chunks for a query embedding in milliseconds.

Metadata filtering narrows retrieval by structured attributes (date, source, department, access level) alongside the vector search — critical for enterprise use cases with permissioning or freshness requirements.

5.4 Hybrid Search, Re-ranking & Context Compression

- **Hybrid search** combines dense vector similarity with sparse keyword methods (BM25), catching exact-match cases (IDs, acronyms, product codes) that pure embedding similarity can miss.
- **Re-ranking** passes the initial candidate set through a more expensive, more accurate cross-encoder model to re-score and reorder results before they reach the LLM — trading a small latency cost for a meaningful precision gain.
- **Context compression** trims or summarizes retrieved passages to keep only the most relevant sentences, reducing token usage and helping the model focus on what matters within a limited context window.

5.5 Agentic RAG

Agentic RAG extends the static "retrieve-then-generate" pattern into a loop where the model itself decides whether to retrieve, reformulate the query, retrieve again, call external tools/APIs, or answer — rather than always executing one fixed retrieval step per query. This lets a system handle multi-hop questions (requiring several rounds of lookup), self-correct on poor initial retrieval, and blend RAG with function calling and multi-step planning for genuinely complex enterprise workflows.

Engineering Note

Agentic loops trade predictable latency/cost for flexibility — each additional retrieval or tool round adds a full model call. Production systems typically cap the number of loop iterations and log every step for observability and debugging.

Evaluation, Deployment & Best Practices

6.1 Evaluation Metrics

TRADITIONAL NLP METRICS

Still relevant for classification-style or extraction sub-tasks within a larger LLM system:

Metric	Measures
Accuracy	Fraction of predictions that exactly match the expected label/answer.
Precision	Of the items predicted positive, the fraction that are actually positive.
Recall	Of the items that are actually positive, the fraction the model correctly identified.
F1	Harmonic mean of precision and recall — a single balanced score.

LLM / RAG-SPECIFIC METRICS

Metric	Measures
Groundedness	Whether the answer's claims are actually supported by the retrieved context, rather than the model's parametric memory.
Faithfulness	Whether the answer avoids contradicting or misrepresenting the source material provided.
Context Relevance	Whether the chunks retrieved are actually relevant to the query — a retriever-quality metric independent of the LLM.
Answer Relevance	Whether the generated answer actually addresses what the user asked.
Metadata Filtering Success	Whether retrieval correctly honored structured filters (date range, source, permissions) requested or implied by the query.
Hallucination Rate	Frequency of confidently stated but unsupported or fabricated claims — typically measured by human review or an LLM-as-judge pipeline against source documents.

PERFORMANCE METRICS

LATENCY

Total wall-clock time from request to complete response.

TOKENS PER SECOND

Generation speed once streaming has started — affects how "fast" a response feels to read.

TIME TO FIRST TOKEN (TTFT)

Delay before the first streamed token arrives — the dominant factor in perceived responsiveness for chat UIs.

COST PER QUERY

Total input + output token cost (plus retrieval/embedding cost in RAG systems) per request — the key unit economics metric.

THROUGHPUT

Requests (or tokens) the serving infrastructure can process per second across all concurrent users.

TOKEN USAGE

Input and output token counts tracked per request for cost attribution, quota management, and context-budget planning.

6.2 Production Best Practices & Security

Prompt Injection

Untrusted content (web pages, documents, user input, tool outputs) can contain instructions designed to hijack the model's behavior. Mitigate with strict system/user role separation, input sanitization, output validation, and treating any retrieved or tool-returned content as data, never as instructions.

Security

Apply least-privilege API keys, rate limiting, PII redaction on both input and output, encrypt data in transit/at rest, and audit which internal systems/tools an agent is permitted to call — every tool binding is an expansion of attack surface.

Context Window Management

Budget tokens deliberately across system prompt, retrieved context, conversation history, and expected output. Truncate or summarize aging conversation turns; prioritize the most relevant retrieved chunks when the window is constrained.

Model Selection

Match model size/capability to task difficulty — routing simple classification or extraction to smaller, cheaper, faster models and reserving frontier models for complex reasoning materially reduces cost and latency at scale.

Monitoring & Logging

Log prompts, responses, retrieved context, latency, token usage, and user feedback for every request (respecting data-handling policy). This is the raw material for regression detection, cost analysis, and offline evaluation.

Guardrails

Layer input/output content filters, schema validation for structured outputs, refusal handling, and human-in-the-loop review for high-stakes actions (e.g., anything that sends money, deletes data, or contacts a customer) before it executes.

CHAPTER SUMMARY

- Evaluate both retrieval quality and generation quality separately — a good LLM cannot fix bad retrieval, and good retrieval cannot fix a poorly grounded generation step.
- Track performance and cost metrics from day one; they are far harder to retrofit than to instrument up front.
- Treat every untrusted input — user text, retrieved documents, tool responses — as a potential prompt-injection vector.

Mathematical Foundations

The minimum math vocabulary needed to read Transformer papers and code without translation — concepts only, no derivations.

VECTORS & MATRICES

A token embedding is a **vector** — an ordered list of numbers in d -dimensional space. Stacking vectors for every token in a sequence forms a **matrix**. Nearly every Transformer operation (attention, FFN) is matrix multiplication between learned weight matrices and these token representations — the reason GPUs, built for parallel matrix math, are the right hardware for this workload.

DOT PRODUCT & COSINE SIMILARITY

The **dot product** of two vectors measures how aligned they are — large when vectors point in similar directions with large magnitude. Self-attention's $Q \cdot K^T$ step is exactly this: a similarity score between every query and every key. **Cosine similarity** normalizes the dot product by vector length, isolating direction (semantic similarity) from magnitude — the standard metric for comparing embeddings in vector search.

SOFTMAX

$$\text{softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

Converts an arbitrary vector of real numbers (logits) into a probability distribution: every value becomes positive and the whole vector sums to 1, while preserving the relative ranking of the inputs.

GRADIENT DESCENT & BACKPROPAGATION

Training minimizes a **loss function** (how wrong the model's predictions are) by repeatedly nudging every parameter in the direction that reduces loss fastest — the negative gradient.

Backpropagation is the algorithm that efficiently computes this gradient for every parameter in a deep network by applying the chain rule of calculus layer by layer, from output back to input.

CROSS-ENTROPY LOSS

The standard loss for next-token prediction: it penalizes the model in proportion to how much probability mass it placed on the *wrong* token instead of the actual next token in the training data — driving the model to assign higher probability to correct continuations over time.

PROBABILITY DISTRIBUTIONS

An LLM's output at each generation step is not a single answer but a **probability distribution** over the entire vocabulary. Everything in Chapter 4.3 — temperature, top-k, top-p — is a strategy for turning that distribution into one concrete chosen token.

SCALING LAWS

Empirical relationships (e.g., Kaplan et al., Chinchilla) showing that model loss decreases smoothly and predictably as a power law in three quantities: parameter count, dataset size, and compute — the quantitative basis for deciding how large a model to train and on how much data, given a fixed compute budget.

End of handbook. For architecture-specific deep dives (e.g., mixture-of-experts, grouped-query attention, RoPE variants) or provider-specific API references, consult the relevant model card or API documentation, as these details evolve quickly and vary by vendor.